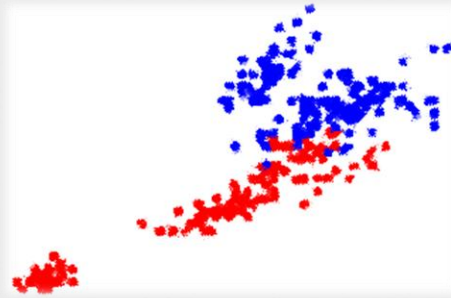
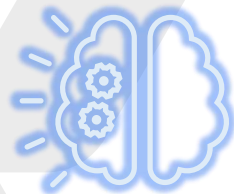


1010
1010

k Nearest Neighbors

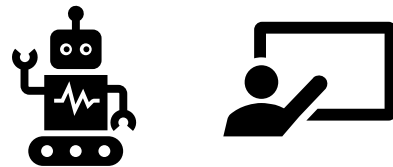
k-NN



UNE APPROCHE DU MACHINE LEARNING

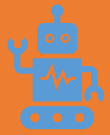
Algorithme des K PLUS PROCHES VOISINS

A propos du Machine Learning



Introduction au Machine Learning

A propos du Machine Learning



Le machine learning (apprentissage automatique) est un domaine de l'intelligence artificielle.



Ce domaine utilise des données en masse (big data) afin de produire des cycles d'apprentissage à une machine implémentant des algorithmes prévus à cet effet.



Le processus devient efficace qu'à l'issue de la phase d'apprentissage.

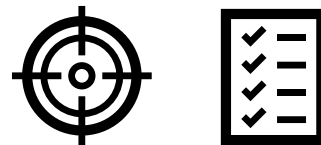
A propos du Machine Learning

- On distingue plusieurs types d'apprentissage :
 - Apprentissage supervisé :
 - Les données d'apprentissage ont été traitées en amont afin de les labeliser/étiqueter. C'est-à-dire que les différentes données sont regroupées dans des classes.
 - Apprentissage non supervisé :
 - Les données ne sont pas initialement catégorisées. C'est à la machine et son algorithme de produire du classement durant le processus.
- Et d'autres encore :
 - Apprentissage semi-supervisé / partiellement supervisé / apprentissage par renforcement / apprentissage par transfert.

A propos du Machine Learning

- Quelques algorithmes utilisés dans le machine learning :
 - **Les réseaux de neurones** dont fait partie les méthodes dites d'apprentissage profond (deep learning) qui réalise de l'apprentissage supervisé ou non supervisé
 - **La méthode des k plus proches voisins**, algorithme qui fait l'objet de notre TP, c'est un algorithme d'apprentissage supervisé.
 - **Les algorithmes génétiques** : Permet d'obtenir une solution approchée dans un problème d'optimisation ne pouvant pas être résolu par une méthodologie exacte. Ces algorithmes s'inspirent de l'évolution de Darwin (sélection naturelle). La sélection est appliquée à plusieurs solutions potentielle du problème. On sélectionne les meilleurs par bonds/génération successives.

Objectif de l'algorithme **k-NN** (**k** Nearest **N**eighbors)



Entrées & Résultats Produits

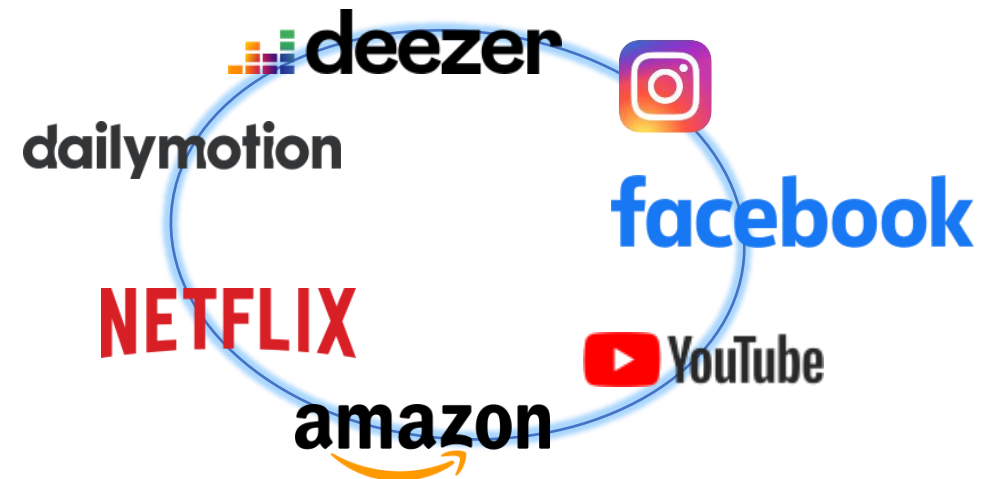
Objectif de l'algorithme k-NN

- L'algorithme des k plus proches voisins est un algorithme d'apprentissage automatique de type **supervisé**.
- Il fonctionne avec un jeu des données qui ont été préalablement **labelisées**, (on peut dire aussi **classée**).
- Il peut produire deux types de résultats selon le problème initialement posé :
 - **Classer** une nouvelle valeur dans l'une des classes du jeu de données labelisées.
 - **Attribuer** une valeur (par **régression***) à une nouvelle donnée (on n'abordera pas cette catégorie de problèmes dans cette activité).

***Régression :** Méthode d'analyse statique permettant de déterminer une valeur à partir d'autres valeurs corrélées (= il existe un lien entre les différentes valeurs).

Objectif de l'algorithme k-NN

- Vous utilisez certainement des services en ligne ou connectés de type :
 - Vidéos en streaming : [YouTube](#); [dailymotion](#); etc.
 - Vidéos à la demande **VOD** : [Netflix](#); [Amazon Prime Video](#); etc.
 - Musique en streaming : [Deezer](#); [Spotify](#); etc.
 - Réseaux sociaux : [Facebook](#); [Instagram](#); etc.
 - Achats en ligne : [Amazon](#); etc.
 - Et encore d'autres....



Objectif de l'algorithme **k-NN**

- Toutes ces plateformes disposent d'un service de suggestions :
 - Suggestion de vidéos, de musiques, d'ami(e)s, de produits, etc.
- Ce service de suggestion s'appuie sur l'algorithme **k-NN**.
 - Des profils types (classification) sont établies en amont ;
 - L'algorithme va appliquer le **k-NN** à partir de vos données personnelles : historiques, âge, genre, etc.
 - A l'issue du traitement, vous êtes classifié dans un profil et la plateforme peut vous proposer des suggestions correspondant à votre profil attribué.

Principe de l'algorithme



Etapes et actions

Principe de l'algorithme

- Préambule : Le jeu de données d'apprentissage



Le jeu de données d'apprentissage est un ensemble noté **E** contenant **n** données labellisées ;



Chacune des données est donc **classée** selon son **label** ;



Chaque donnée est constituée de **p** valeurs/variables dites **prédicatrices**.

Principe de l'algorithme

- Description de l'algorithme :



Pour chacune des données d'apprentissage, calculer sa distance par rapport à la donnée testée **u**;



On retient les **k** données du jeu d'apprentissage ayant la plus petite distance ;

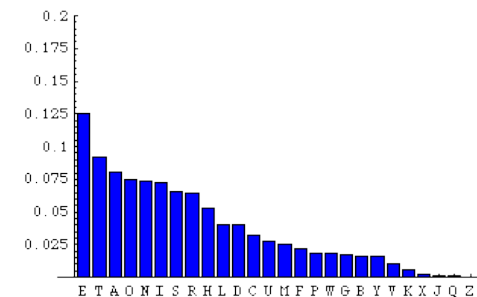


On attribue à **u** la classe qui est la plus fréquente dans les **k** données.

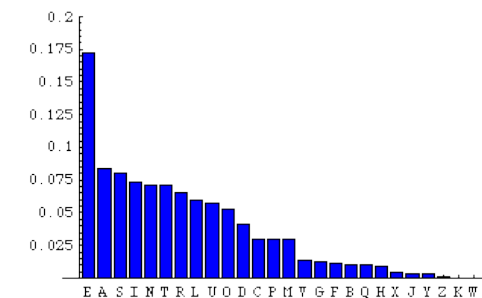
Principe de l'algorithme

- Illustration :

- On désire classer un texte/récit selon sa langue, c'est la donnée à traiter.
- Les variables prédictives sont la **fréquence** d'apparition des lettres de l'alphabet dans différents textes.
- Les labels sont les langues d'expression des textes de la base d'apprentissage.



Fréquence en anglais
dans un texte n°i



Fréquence en français
dans un texte n°j

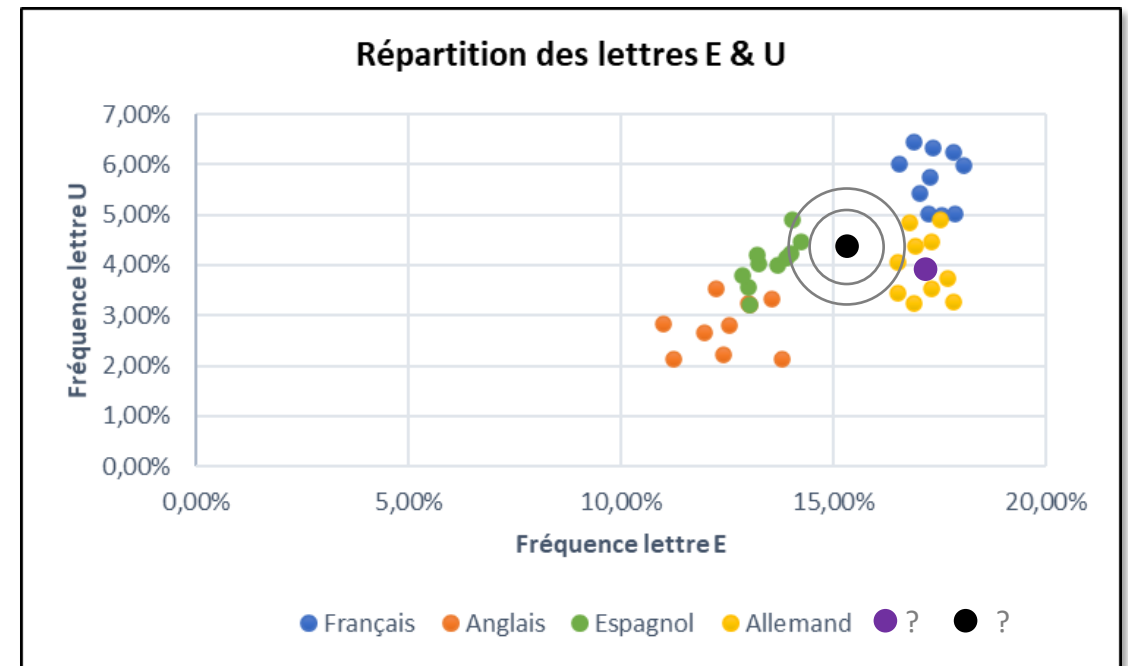
- Illustration :

- ### Repartition des lettres E & U
-
- The scatter plot displays the relationship between the frequency of the letter 'E' (x-axis) and the letter 'U' (y-axis) for four languages. The x-axis ranges from 0,00% to 20,00% with major grid lines every 5,00%. The y-axis ranges from 0,00% to 7,00% with major grid lines every 1,00%. The data points are clustered by language: Français (blue) is at the top right, Espagnol (green) is in the middle, Allemand (yellow) is at the bottom right, and Anglais (orange) is at the bottom left. The legend at the bottom identifies the colors for each language.
- | Langue | Fréquence lettre E (%) | Fréquence lettre U (%) |
|----------|------------------------|------------------------|
| Français | 17,00 | 6,00 |
| Français | 17,50 | 6,50 |
| Français | 18,00 | 6,00 |
| Français | 18,00 | 6,20 |
| Français | 18,50 | 5,00 |
| Français | 18,50 | 6,00 |
| Français | 19,00 | 6,00 |
| Français | 19,00 | 6,20 |
| Français | 19,50 | 6,00 |
| Français | 19,50 | 6,20 |
| Français | 20,00 | 6,00 |
| Français | 20,00 | 6,20 |
| Français | 20,50 | 6,00 |
| Français | 20,50 | 6,20 |
| Français | 21,00 | 6,00 |
| Français | 21,00 | 6,20 |
| Français | 21,50 | 6,00 |
| Français | 21,50 | 6,20 |
| Français | 22,00 | 6,00 |
| Français | 22,00 | 6,20 |
| Français | 22,50 | 6,00 |
| Français | 22,50 | 6,20 |
| Français | 23,00 | 6,00 |
| Français | 23,00 | 6,20 |
| Français | 23,50 | 6,00 |
| Français | 23,50 | 6,20 |
| Français | 24,00 | 6,00 |
| Français | 24,00 | 6,20 |
| Français | 24,50 | 6,00 |
| Français | 24,50 | 6,20 |
| Français | 25,00 | 6,00 |
| Français | 25,00 | 6,20 |
| Français | 25,50 | 6,00 |
| Français | 25,50 | 6,20 |
| Français | 26,00 | 6,00 |
| Français | 26,00 | 6,20 |
| Français | 26,50 | 6,00 |
| Français | 26,50 | 6,20 |
| Français | 27,00 | 6,00 |
| Français | 27,00 | 6,20 |
| Français | 27,50 | 6,00 |
| Français | 27,50 | 6,20 |
| Français | 28,00 | 6,00 |
| Français | 28,00 | 6,20 |
| Français | 28,50 | 6,00 |
| Français | 28,50 | 6,20 |
| Français | 29,00 | 6,00 |
| Français | 29,00 | 6,20 |
| Français | 29,50 | 6,00 |
| Français | 29,50 | 6,20 |
| Français | 30,00 | 6,00 |
| Français | 30,00 | 6,20 |
| Français | 30,50 | 6,00 |
| Français | 30,50 | 6,20 |
| Français | 31,00 | 6,00 |
| Français | 31,00 | 6,20 |
| Français | 31,50 | 6,00 |
| Français | 31,50 | 6,20 |
| Français | 32,00 | 6,00 |
| Français | 32,00 | 6,20 |
| Français | 32,50 | 6,00 |
| Français | 32,50 | 6,20 |
| Français | 33,00 | 6,00 |
| Français | 33,00 | 6,20 |
| Français | 33,50 | 6,00 |
| Français | 33,50 | 6,20 |
| Français | 34,00 | 6,00 |
| Français | 34,00 | 6,20 |
| Français | 34,50 | 6,00 |
| Français | 34,50 | 6,20 |
| Français | 35,00 | 6,00 |
| Français | 35,00 | 6,20 |
| Français | 35,50 | 6,00 |
| Français | 35,50 | 6,20 |
| Français | 36,00 | 6,00 |
| Français | 36,00 | 6,20 |
| Français | 36,50 | 6,00 |
| Français | 36,50 | 6,20 |
| Français | 37,00 | 6,00 |
| Français | 37,00 | 6,20 |
| Français | 37,50 | 6,00 |
| Français | 37,50 | 6,20 |
| Français | 38,00 | 6,00 |
| Français | 38,00 | 6,20 |
| Français | 38,50 | 6,00 |
| Français | 38,50 | 6,20 |
| Français | 39,00 | 6,00 |
| Français | 39,00 | 6,20 |
| Français | 39,50 | 6,00 |
| Français | 39,50 | 6,20 |
| Français | 40,00 | 6,00 |
| Français | 40,00 | 6,20 |
| Français | 40,50 | 6,00 |
| Français | 40,50 | 6,20 |
| Français | 41,00 | 6,00 |
| Français | 41,00 | 6,20 |
| Français | 41,50 | 6,00 |
| Français | 41,50 | 6,20 |
| Français | 42,00 | 6,00 |
| Français | 42,00 | 6,20 |
| Français | 42,50 | 6,00 |
| Français | 42,50 | 6,20 |
| Français | 43,00 | 6,00 |
| Français | 43,00 | 6,20 |
| Fran | | |

Principe de l'algorithme

- **Illustration :**

- Ajoutons un 1^{er} texte (point violet) de langue inconnue.
- Dans ce cas l'analyse visuelle du graphique est immédiate.
- Ajoutons un 2^e texte (point noir) de langue inconnue.
- L'analyse est moins aisée.

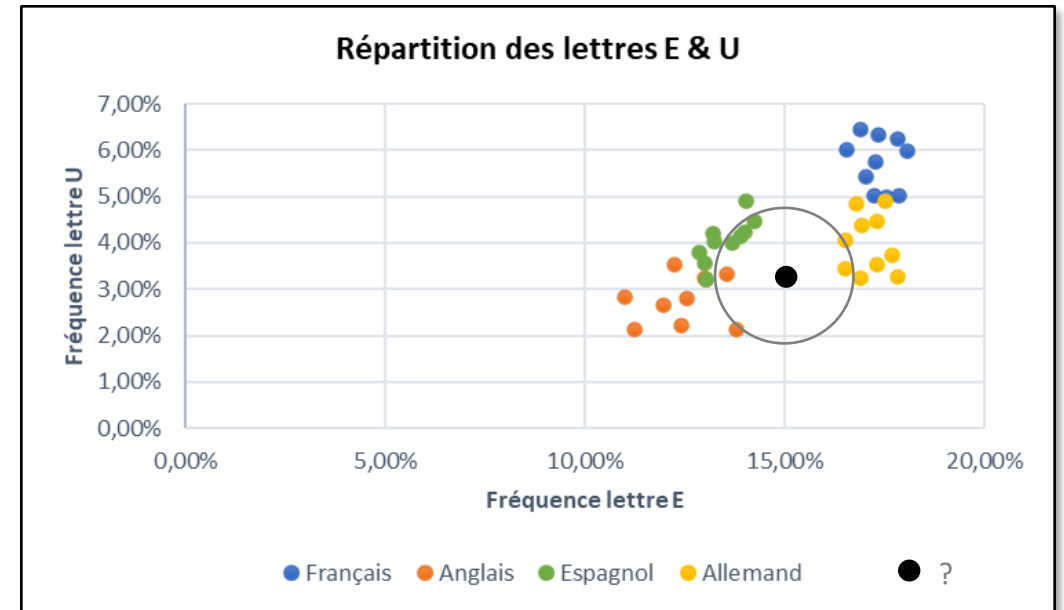


Principe de l'algorithme

- A propos de la fonction de distance :

- Il existe plusieurs fonctions pour calculer la distance entre le point de données et les **k** données voisines.
- La plus courante et facile à appréhender est la **distance euclidienne**.
- En géométrie plane (2D) la distance euclidienne a pour formule : $D = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- Appliquer à notre étude avec les lettres **U** et **E** :

- $D = \sqrt{(x_{i/5} - x_{5/u})^2 + (x_{i/21} - x_{u/21})^2}$
 - i : numéro du texte
 - u : texte à classer
 - $x_{i/5}$: Fréquence de la 5^{ème} lettres (E) du texte n°i
 - $x_{u/5}$: Fréquence de la 5^{ème} lettre (E) du texte à classer



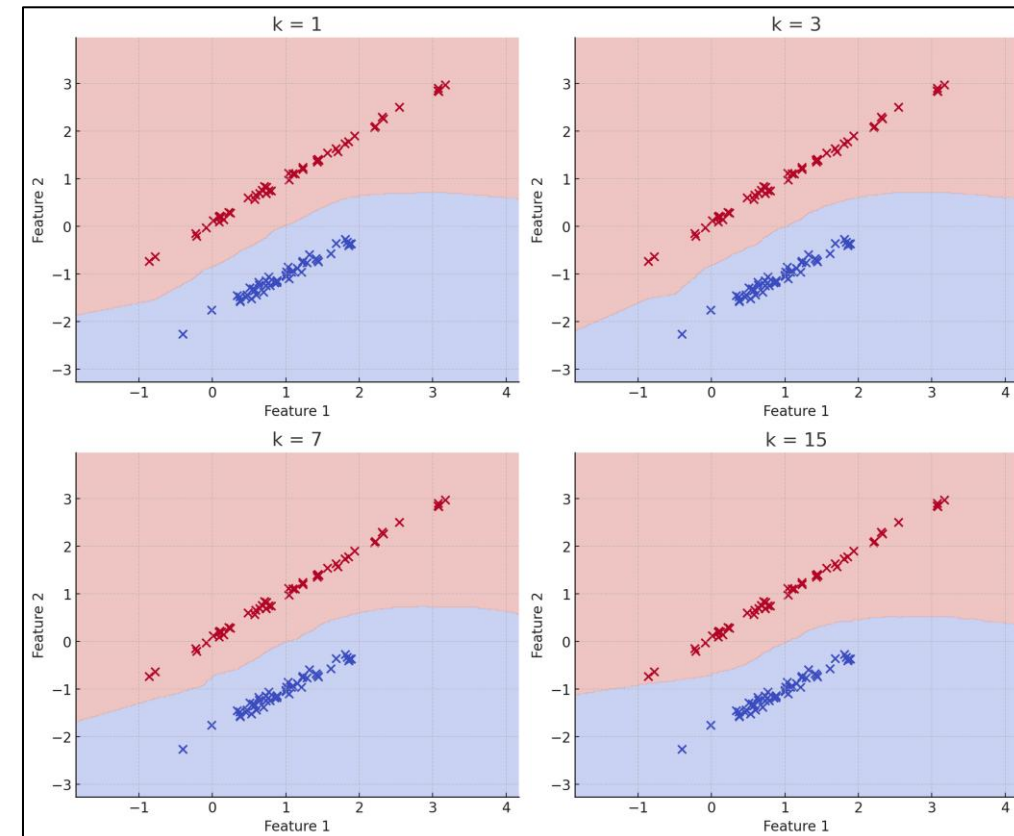
Principe de l'algorithme

- **A propos de la fonction de distance :**

- Nous étions limités à 2 lettres, mais on peut appliquer le calcul aux 26 lettres !
- La fonction de calcul de la distance euclidienne peut s'appliquer à **p** dimensions !
- La formule générale est donc : $D = \sqrt{\sum_{j=1}^p (x_{i/j} - x_{u/j})^2}$

Principe de l'algorithme

- Le choix de la valeur de k :
 - Le choix de la valeur de k n'est pas aisé.
 - Le classement peut être modifié selon la valeur de k adopté.
 - Un k trop petit ne permet de profiter de l'apprentissage.
 - Mais inversement un k trop grand risque d'occulter des classes.
- Le nombre n de données d'apprentissage :
 - La pertinence des résultats augmente avec le nombre de données d'apprentissage.
 - Cependant le coût algorithmique augmente également.



Configuration de votre poste

Installation des bibliothèques de traitement et d'affichage des courbes

Bibliothèques – Présentation

- Pour mener notre étude nous allons utiliser 3 bibliothèques :
 - Pandas qui nous permettra d'importer les données issues du fichier csv
 - Matplotlib qui va nous permettre de visualiser les données (tracer des graphiques)
 - Scikit-learn qui propose une implémentation de l'algorithme des **k** plus proches voisins.



Bibliothèques – Installation

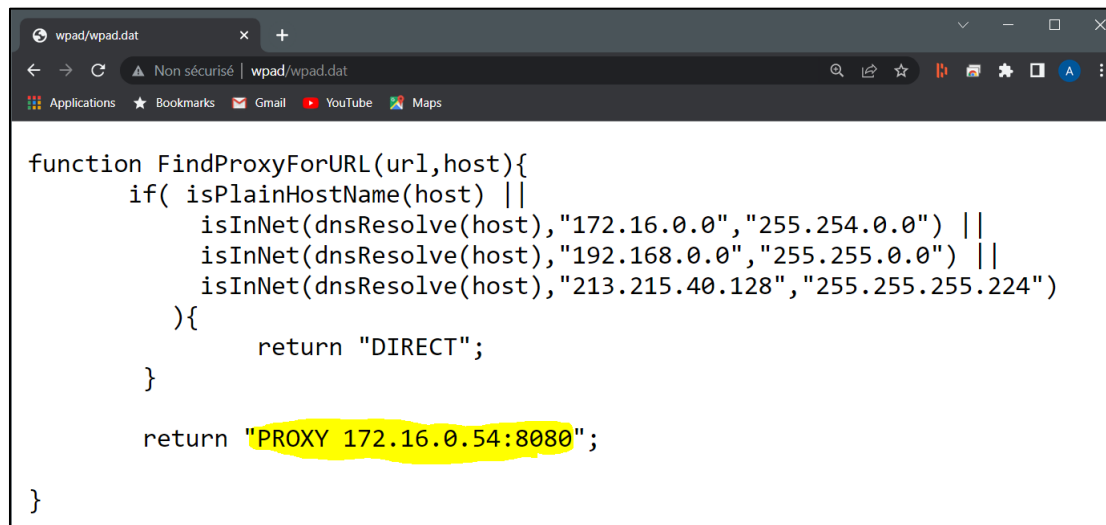
- A propos de PIP et PyPI :

- **Python** est généralement installé avec **PIP**.
- **PIP** (Pip Installs Packages) est le système de gestion de paquets standard pour **Python**. C'est un outil en ligne de commande qui permet d'installer, désinstaller et gérer des paquets Python tiers.
- Une bibliothèque étant constituée d'un ou plusieurs paquets, **PIP** permet d'installer indifféremment des paquets ou des bibliothèques.
- **PyPI** (**Python Package Index**) est le dépôt officiel des paquets **Python**, il a donc pour objectif d'accueillir et recenser les paquets proposés par la communauté pour les proposer à tous via une simple connexion Internet.



Bibliothèques – Installation

- **Etape 1** : Récupération de l'IP du serveur PROXY pour PIP
 - Toute connexion à Internet à partir du lycée transite par un serveur PROXY, nous aurons besoin de renseigner l'adresse IP et le port du PROXY à **PIP** pour qu'il puisse récupérer les paquets sur **PyPI**.
 - Pour ce faire, ouvrir un navigateur et saisir l'URL : <http://wpad/wpad.dat>
 - Vous trouverez l'adresse et le port sur la réponse comme ci-dessous :

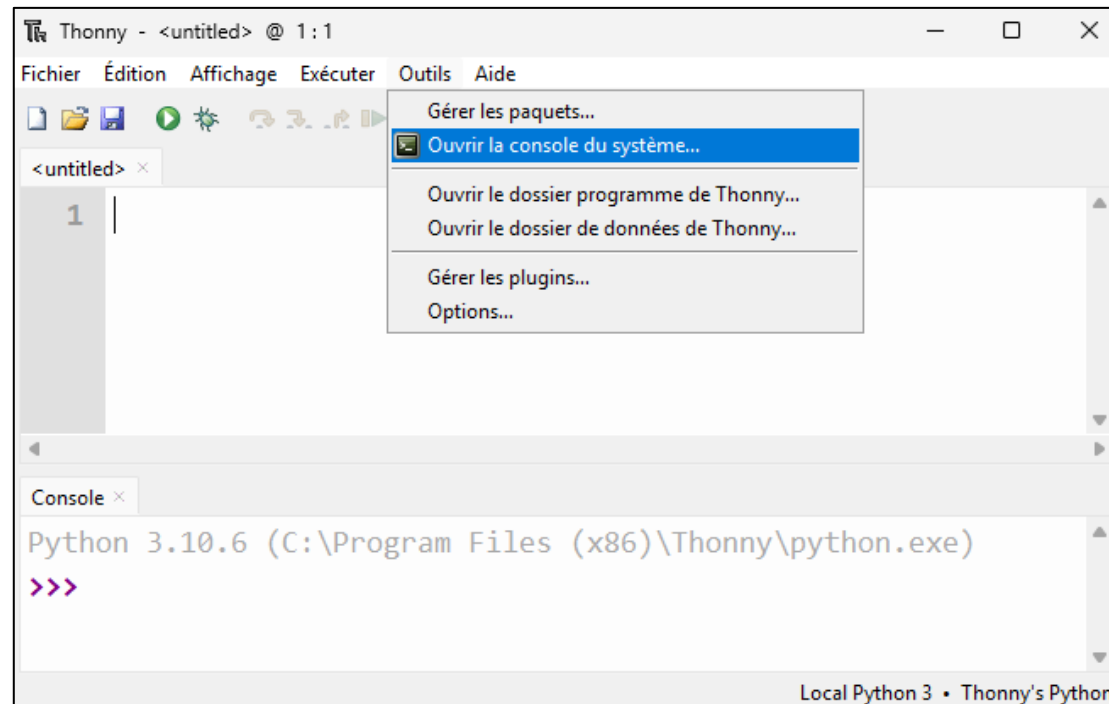


```
function FindProxyForURL(url,host){
    if( isPlainHostName(host) ||
        isInNet(dnsResolve(host),"172.16.0.0","255.254.0.0") ||
        isInNet(dnsResolve(host),"192.168.0.0","255.255.0.0") ||
        isInNet(dnsResolve(host),"213.215.40.128","255.255.255.224")
    ){
        return "DIRECT";
    }

    return "PROXY 172.16.0.54:8080";
}
```

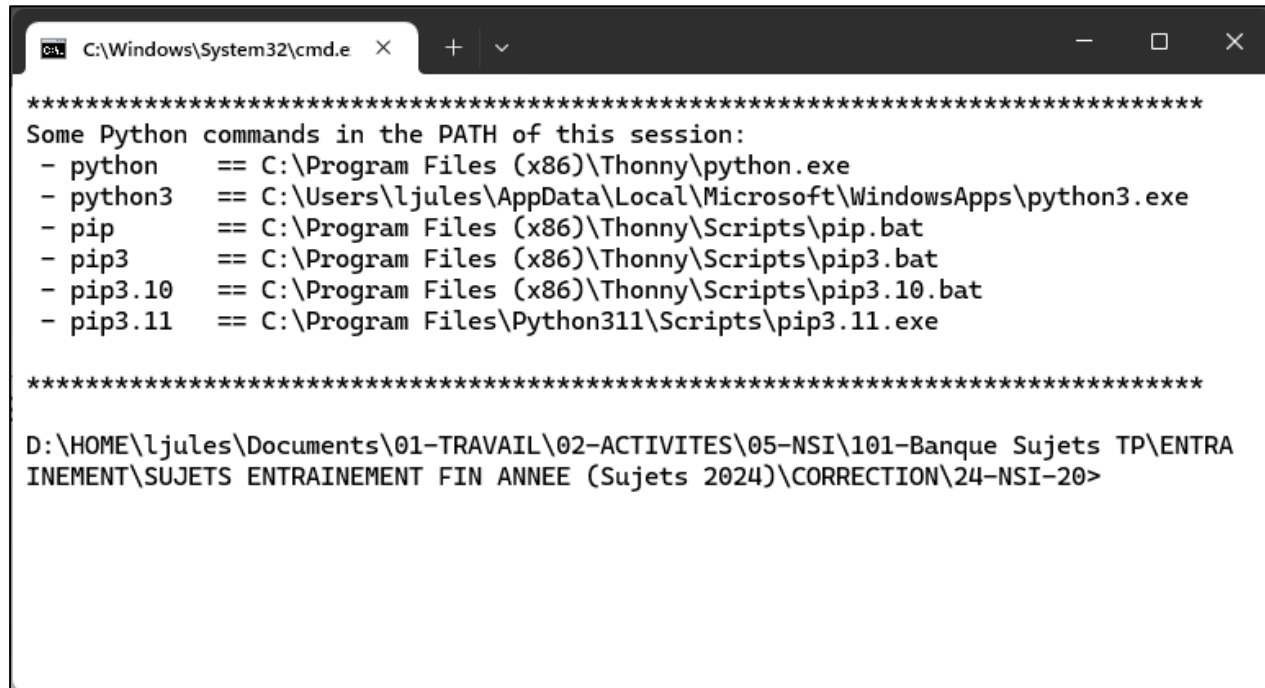
Bibliothèques – Installation

- **Etape 2 :** Ouverture de la console via **Thonny**
 - Ouvrir **Thonny** et cliquer sur *Outils/ouvrir la console du système...*



Bibliothèques – Installation

- **Etape 2 : Ouverture de la console via Thonny**
 - La console ainsi obtenue a été automatique configurée par **Thonny** afin de travailler directement avec l'environnement d'installation de **Thonny**.



```
cmd C:\Windows\System32\cmd.e X + - □ X
*****
Some Python commands in the PATH of this session:
- python == C:\Program Files (x86)\Thonny\python.exe
- python3 == C:\Users\ljules\AppData\Local\Microsoft\WindowsApps\python3.exe
- pip == C:\Program Files (x86)\Thonny\Scripts\pip.bat
- pip3 == C:\Program Files (x86)\Thonny\Scripts\pip3.bat
- pip3.10 == C:\Program Files (x86)\Thonny\Scripts\pip3.10.bat
- pip3.11 == C:\Program Files\Python311\Scripts\pip3.11.exe
*****
D:\HOME\ljules\Documents\01-TRAVAIL\02-ACTIVITES\05-NSI\101-Banque Sujets TP\ENTRA
INEMENT\SUJETS ENTRAINEMENT FIN ANNEE (Sujets 2024)\CORRECTION\24-NSI-20>
```


Bibliothèques – Installation

- Etape 3 : Installation des bibliothèques



- Installation de **Pandas** :

- Exécuter la commande : `pip install --proxy=172.16.0.54:8080 --user pandas`

```
Collecting pandas
  Downloading pandas-2.2.3-cp310-cp310-win_amd64.whl (11.6 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.6/11.6 MB 43.7 MB/s eta 0:00:00
Requirement already satisfied: pytz>=2020.1 in c:\users\ljules\appdata\roaming\python\python310\site-packages (from pandas) (2023.3)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\ljules\appdata\roaming\python\python310\site-packages (from pandas) (2.8.2)
Collecting numpy>=1.22.4
  Downloading numpy-2.2.5-cp310-cp310-win_amd64.whl (12.9 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.9/12.9 MB 54.7 MB/s eta 0:00:00
Collecting tzdata>=2022.7
  Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 347.8/347.8 kB 21.1 MB/s eta 0:00:00
Requirement already satisfied: six>=1.5 in c:\program files (x86)\thonny\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Installing collected packages: tzdata, numpy, pandas
Successfully installed numpy-2.2.5 pandas-2.2.3 tzdata-2025.2
```

Bibliothèques – Installation

- Etape 3 : Installation des bibliothèques



- Installation de **matplotlib** :

- Exécuter la commande : `pip install --proxy=172.16.0.54:8080 --user matplotlib`

```
Collecting matplotlib
  Downloading matplotlib-3.10.1-cp310-cp310-win_amd64.whl (8.1 MB)
    8.1/8.1 MB 27.1 MB/s eta 0:00:00
Collecting kiwisolver>=1.3.1
  Downloading kiwisolver-1.4.8-cp310-cp310-win_amd64.whl (71 kB)
    71.9/71.9 kB ? eta 0:00:00
Collecting packaging>=20.0
  Downloading packaging-25.0-py3-none-any.whl (66 kB)
    66.5/66.5 kB ? eta 0:00:00
Collecting fonttools>=4.22.0
  Downloading fonttools-4.57.0-cp310-cp310-win_amd64.whl (2.2 MB)
    2.2/2.2 MB 46.7 MB/s eta 0:00:00
Collecting cycler>=0.10
  Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Collecting pyparsing>=2.3.1
  Downloading pyparsing-3.2.3-py3-none-any.whl (111 kB)
    111.1/111.1 kB 6.7 MB/s eta 0:00:00
Collecting contourpy>=1.0.1
  Downloading contourpy-1.3.2-cp310-cp310-win_amd64.whl (221 kB)
    221.2/221.2 kB 13.2 MB/s eta 0:00:00
Requirement already satisfied: python-dateutil>=2.7 in c:\users\ljules\appdata\roaming\python\python310\site-packages (from matplotlib) (2.8.2)
Collecting pillow>=8
  Downloading pillow-11.2.1-cp310-cp310-win_amd64.whl (2.7 MB)
    2.7/2.7 MB 42.3 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.23 in c:\users\ljules\appdata\roaming\python\python310\site-packages (from matplotlib) (2.2.5)
Requirement already satisfied: six>=1.5 in c:\program files (x86)\thonny\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Installing collected packages: pyparsing, pillow, packaging, kiwisolver, fonttools, cycler, contourpy, matplotlib
Successfully installed contourpy-1.3.2 cycler-0.12.1 fonttools-4.57.0 kiwisolver-1.4.8 matplotlib-3.10.1 packaging-25.0 pillow-11.2.1 pyparsing-3.2.3
```

Bibliothèques – Installation



- **Etape 3** : Installation des bibliothèques

- Installation de **scikit-learn** :

- Exécuter la commande : `pip install --proxy=172.16.0.54:8080 --user scikit-learn`

```
Collecting scikit-learn
  Downloading scikit_learn-1.6.1-cp310-cp310-win_amd64.whl (11.1 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.1/11.1 MB 40.9 MB/s eta 0:00:00
Collecting joblib>=1.2.0
  Downloading joblib-1.4.2-py3-none-any.whl (301 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 301.8/301.8 kB 19.4 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.19.5 in c:\users\ljules\appdata\roaming\python\python310\site-packages (from scikit-learn) (2.2.5)
Collecting threadpoolctl>=3.1.0
  Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Collecting scipy>=1.6.0
  Downloading scipy-1.15.2-cp310-cp310-win_amd64.whl (41.2 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 41.2/41.2 MB 40.9 MB/s eta 0:00:00
Installing collected packages: threadpoolctl, scipy, joblib, scikit-learn
Successfully installed joblib-1.4.2 scikit-learn-1.6.1 scipy-1.15.2 threadpoolctl-3.6.0
```

Application



Classification des iris

Présentation de l'étude

- Support de l'étude :

- Notre jeu de données est connu sous le nom de "**iris de Fisher**". Ce jeu de données a été présentée en 1936 par Ronald Fisher.
- Ce jeu de données est composé de **150 entrées**, pour chaque entrée nous avons :
 - la longueur des sépales (en cm) ;
 - la largeur des sépales (en cm);
 - la longueur des pétales (en cm);
 - la largeur des pétales (en cm);
 - l'espèce d'iris : **Iris setosa**, **Iris virginica** ou **Iris versicolor** (label de classement) .



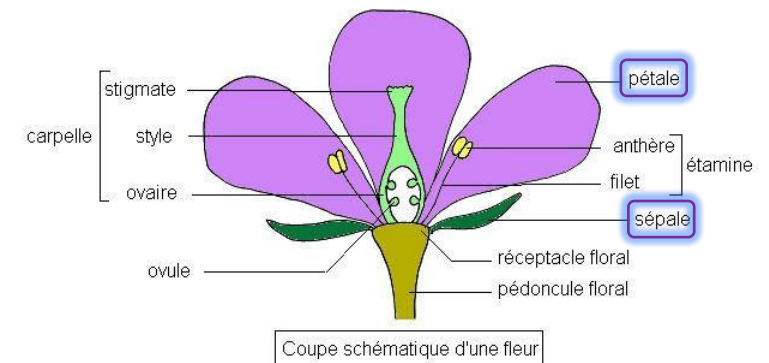
Iris setosa



Iris versicolor



Iris virginica



Préparation du jeu de données

- Ouverture du fichier de jeu de données :

- Ouvrir **Thonny** si ça n'est pas déjà fait
- Ouvrir un nouveau fichier/script Python que vous nommerez : **01-Classification iris.py**
- Copier le script suivant dans votre fichier (les commentaires sont facultatifs) :

```
1 # Import des bibliothèques :
2 # -----
3 import pandas as pd # On utilisera l'abréviation pd pour utiliser Pandas dans notre script
4
5 # Lecture du fichier de données brut :
6 # -----
7
8 # Remarque : Les données tabulaires sont appelées "data frame" dans Pandas, on abrègera "df"
9
10 df_iris = pd.read_csv("IRIS-dataset.csv") # On utilise la méthode de lecture de csv de Panda pour générer un data frame
11 print(df_iris) # On affiche le jeu de données (dataset)
```

Préparation du jeu de données

- Ouverture du fichier de jeu de données :

- Exécuter votre script, vous obtiendrez l'affichage suivant dans la console :

```
>>> %Run 01-Lecture_dataset_iris_et_graphe.py
      sepal_length  sepal_width  petal_length  petal_width  species
0           5.1         3.5         1.4         0.2     setosa
1           4.9         3.0         1.4         0.2     setosa
2           4.7         3.2         1.3         0.2     setosa
3           4.6         3.1         1.5         0.2     setosa
4           5.0         3.6         1.4         0.2     setosa
..          ...         ...         ...         ...         ...
145          6.7         3.0         5.2         2.3  virginica
146          6.3         2.5         5.0         1.9  virginica
147          6.5         3.0         5.2         2.0  virginica
148          6.2         3.4         5.4         2.3  virginica
149          5.9         3.0         5.1         1.8  virginica

[150 rows x 5 columns]
```

Préparation du jeu de données

- Explication :

- **Pandas** affiche les 5 premières et dernières lignes du fichier.
- Il nous indique également le nombre de lignes composant le jeu de données et le nombre de colonnes.

Numéro de ligne	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5
0	sepal_length	sepal_width	petal_length	petal_width	species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[150 rows x 5 columns]

Nombre de lignes et de colonnes du jeu de données

Préparation du jeu de données

- Affichage du jeu de données sous forme de graphe :

- Nous allons tracer un graphe sur lequel nous allons placer :
 - La longueur des pétales en abscisse (axe X);
 - La largeur des pétales en ordonnée (axe Y);
- Nous allons avoir besoin de la bibliothèque **matplotlib** pour réaliser le tracé
- Rajouter l'import de **matplotlib** à votre fichier

```
1 # Import des bibliothèques :
2 # -----
3 import pandas as pd # On utilisera l'abréviation pd pour utiliser Pandas dans notre script
4 import matplotlib.pyplot as plt # On utilisera l'abréviation plt pour utiliser matplotlib
5
```

Préparation du jeu de données

- Affichage du jeu de données sous forme de graphe :

- Nous allons récupérer les valeurs des 2 colonnes : « **sepal_length** » et « **sepal_width** » dans 2 variables **longueur_petale** et **largeur_petale**.
- Ajouter les 2 lignes suivantes à la fin de votre fichier pour procéder à la récupération des valeurs :

```
12 df_iris = pd.read_csv("IRIS-dataset.csv") # On utilise la méthode de lecture de csv de Panda pour générer un data frame
13 print(df_iris) # On affiche le jeu de données (dataset)
14
15
16 # Sélection des données à afficher :
17 # -----
18 longueur_petale = df_iris["petal_length"]
19 largeur_petale = df_iris["petal_width"]
20
```

Préparation du jeu de données

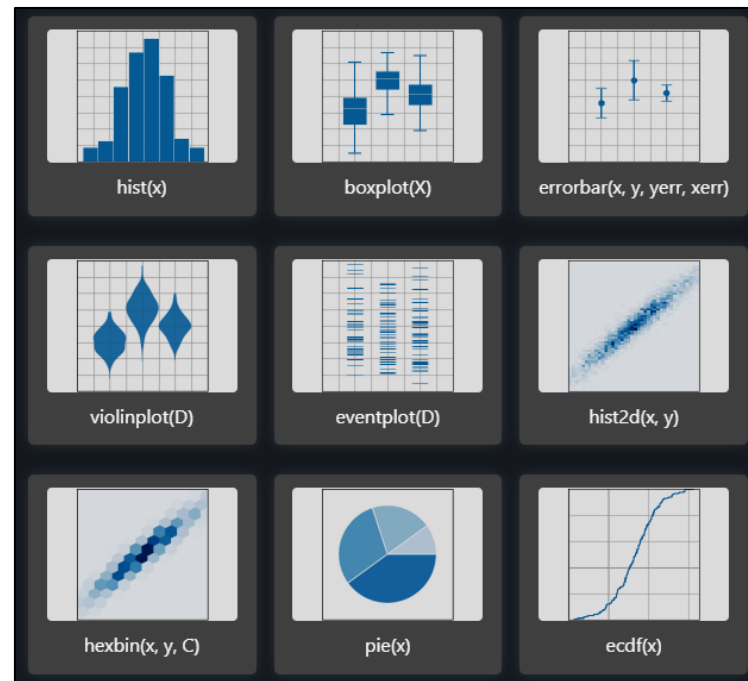
- Affichage du jeu de données sous forme de graphe :

- Nous allons maintenant utiliser **matplotlib** pour afficher nos données la forme d'un graphique « nuage de points ».
- **matplotlib** dispose de la capacité de générer de nombreux types de graphiques :
 - Génération de graphiques 2D pour les données en paires :



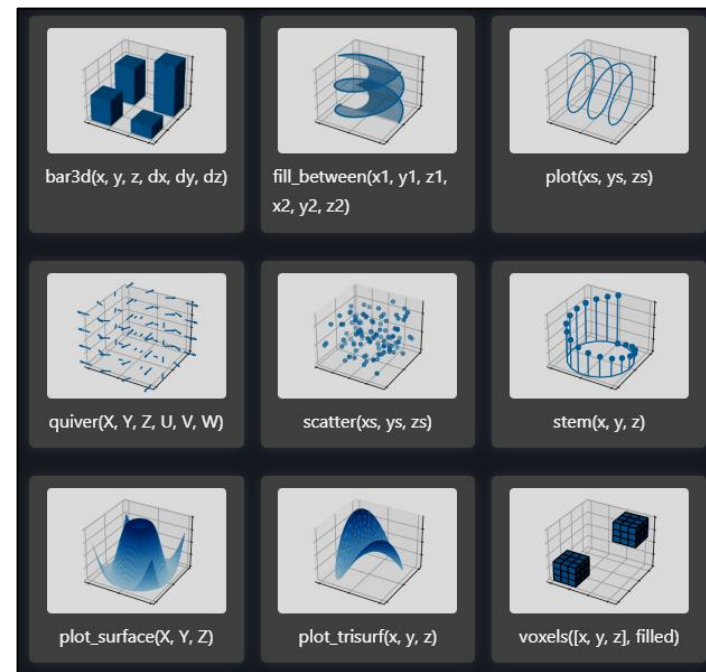
Préparation du jeu de données

- Affichage du jeu de données sous forme de graphe :
 - **matplotlib** dispose de la capacité de générer de nombreux types de graphiques :
 - Génération de graphiques 2D pour les distributions statistiques :



Préparation du jeu de données

- Affichage du jeu de données sous forme de graphe :
 - **matplotlib** dispose de la capacité de générer de nombreux types de graphiques :
 - Génération de graphiques 3D :



Préparation du jeu de données

- Affichage du jeu de données sous forme de graphe :
 - La génération et gestion des graphes se fait à l'aide de méthodes.
 - Pour notre application nous allons utiliser les méthodes suivantes :
 - Afficher un point dans un graphique nuage de points : `plt.scatter(x, y, color='red', marker='o', label='point test')`
 - Afficher des points à partir de listes : `plt.scatter([5, 2, 3, 4],[2, 3, 1, 7])`
 - Ajouter un titre sur l'axe X : `plt.xlabel('Titre de l'axe X')`
 - Ajouter un titre sur l'axe Y : `plt.ylabel('Titre de l'axe Y')`
 - Ajouter un titre sur le graphique : `plt.title('Titre du graphique')`
 - Afficher un texte sur une coordonnée du graphe : `plt.text(3, 2, 'mon texte', fontsize = 10)`
 - Afficher le graphique : `plt.show()`

Préparation du jeu de données

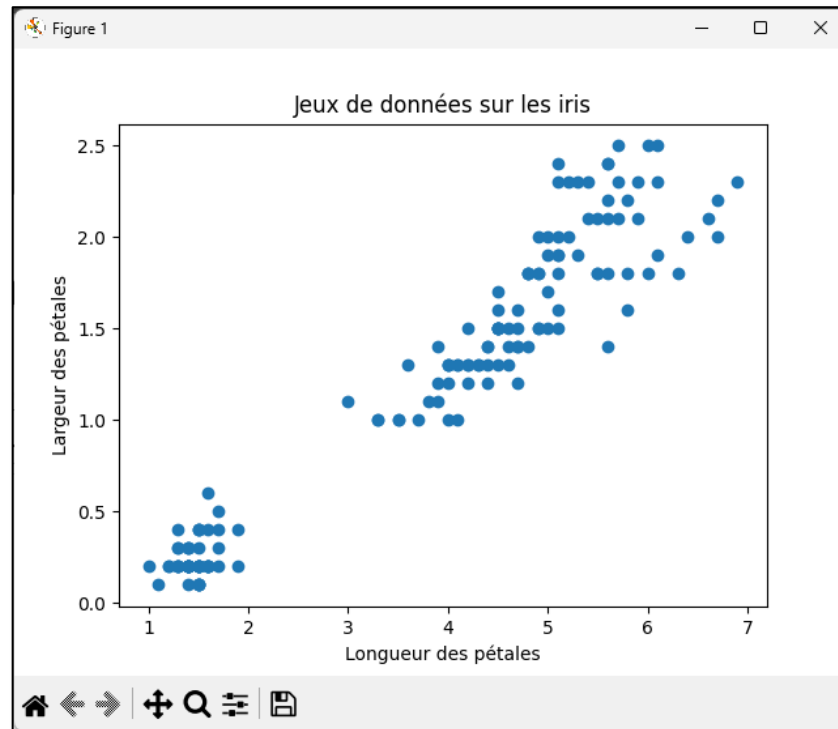
- Affichage du jeu de données sous forme de graphe :

- Préparons l'affichage de notre jeu de données, pour ce faire rajouter le code suivant à la fin de votre script :

```
16 # Sélection des données à afficher :
17 # -----
18 longueur_petale = df_iris["petal_length"]
19 largeur_petale = df_iris["petal_width"]
20
21
22 # Affichage avec matplotlib :
23 # -----
24 plt.title("Jeux de données sur les iris")
25 plt.xlabel("Longueur des pétales")
26 plt.ylabel("Largeur des pétales")
27 plt.scatter(longueur_petale, largeur_petale)
28 plt.show()
```

Préparation du jeu de données

- Affichage du jeu de données sous forme de graphe :
 - Exécuter votre script et vous devriez obtenir le graphique suivant :



Préparation du jeu de données

- Amélioration du graphique :

- Dans notre 1^{er} graphique nous pouvons distinguer que 2 espèces d'iris parmi les 3.
- Afin de faciliter l'analyse, nous allons attribuer une légende et une couleur par espèce.
- Nous allons commencer par récupérer les valeurs de la colonne « species » :
 - Rajouter la ligne : **espece = df_iris["species"]**

```
16 # Sélection des données à afficher :  
17 # -----  
18 longueur_petale = df_iris["petal_length"]  
19 largeur_petale = df_iris["petal_width"]  
20 espece = df_iris["species"]
```

Préparation du jeu de données

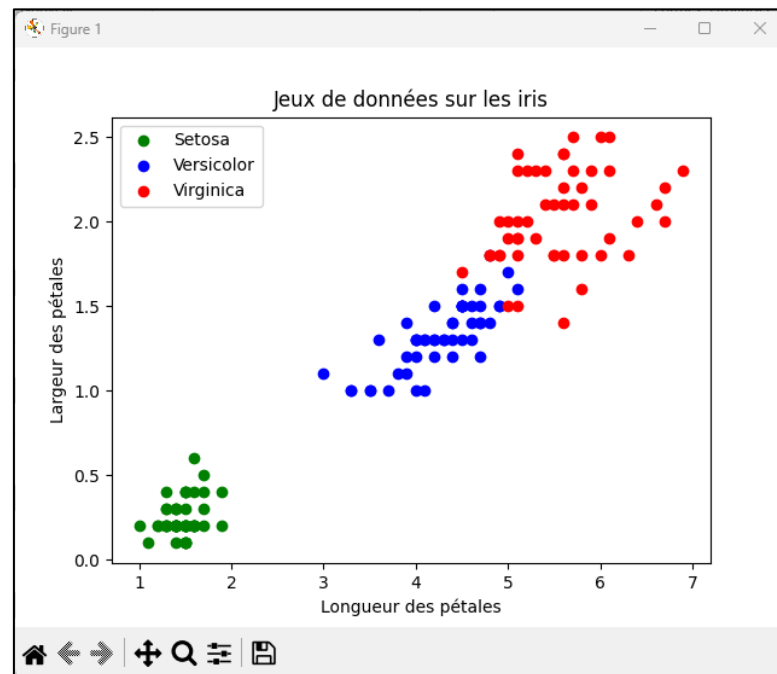
- Amélioration du graphique :

- Nous allons ensuite afficher 3 lots de points, 1 lot par espèce :

```
22
23 # Affichage avec matplotlib :
24 # -----
25 plt.title("Jeux de données sur les iris")
26 plt.xlabel("Longueur des pétales")
27 plt.ylabel("Largeur des pétales")
28 plt.scatter(longueur_petale[espece == "setosa"], largeur_petale[espece == "setosa"], color="green", label="Setosa")
29 plt.scatter(longueur_petale[espece == "versicolor"], largeur_petale[espece == "versicolor"], color="blue", label="Versicolor")
30 plt.scatter(longueur_petale[espece == "virginica"], largeur_petale[espece == "virginica"], color="red", label="Virginica")
31 plt.legend()
32 plt.show()
```

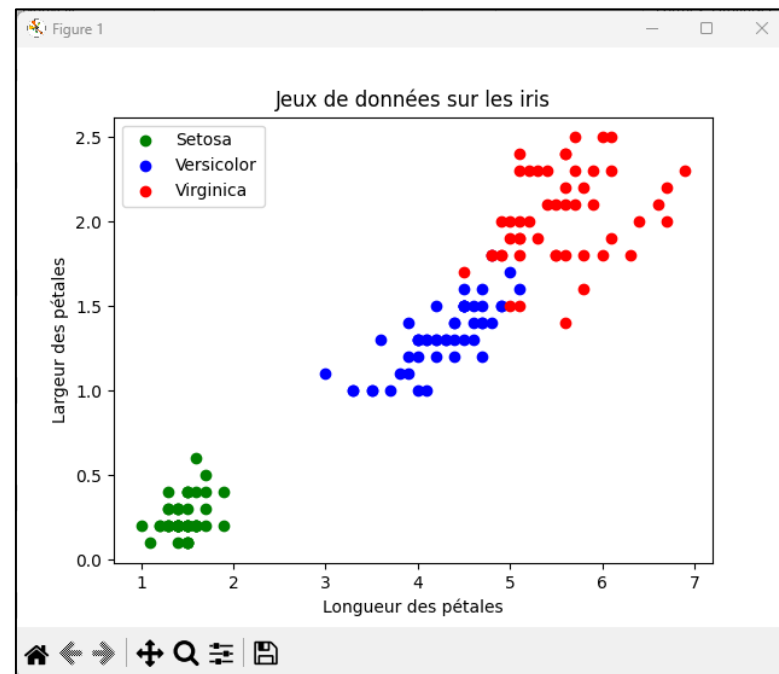
Préparation du jeu de données

- Amélioration du graphique :
 - Vous devriez maintenant obtenir le graphique suivant :



Préparation du jeu de données

- Amélioration du graphique :
 - Vous devriez maintenant obtenir le graphique suivant :



Classification d'une iris inconnue

- **Ajout de la nouvelle iris :**

- Imaginons que vous tombiez sur une iris et que vous souhaitiez déterminer son espèce.
- Nous allons considérer les valeurs suivantes pour la longueur et largeur des pétales :
 - Longueur pétale iris inconnue : 2,5 cm
 - Largeur pétale iris inconnue : 0,75 cm
- Pour procéder au classement, nous pouvons dans un premier temps afficher cette nouvelle iris et voir si elle tombe dans une zone permettant de la classer dans l'une des 3 espèces.

Classification d'une iris inconnue

- Ajout de la nouvelle iris :

- Dans votre fichier Python, rajouter la ligne suivante qui représentera notre nouvelle iris sous la forme d'un dictionnaire contenant en clé/valeur : la longueur et la largeur des pétales.

```
16 # Sélection des données à afficher :
17 # -----
18 longueur_petale = df_iris["petal_length"]
19 largeur_petale = df_iris["petal_width"]
20 espece = df_iris["species"]
21
22
23 # Ajout du nouvel individu :
24 # -----
25 iris_inconnue = {"longueur_pétale": 2.5, "largeur_pétale": 0.75}
26
27
28 # Affichage avec matplotlib :
29 # -----
```

Classification d'une iris inconnue

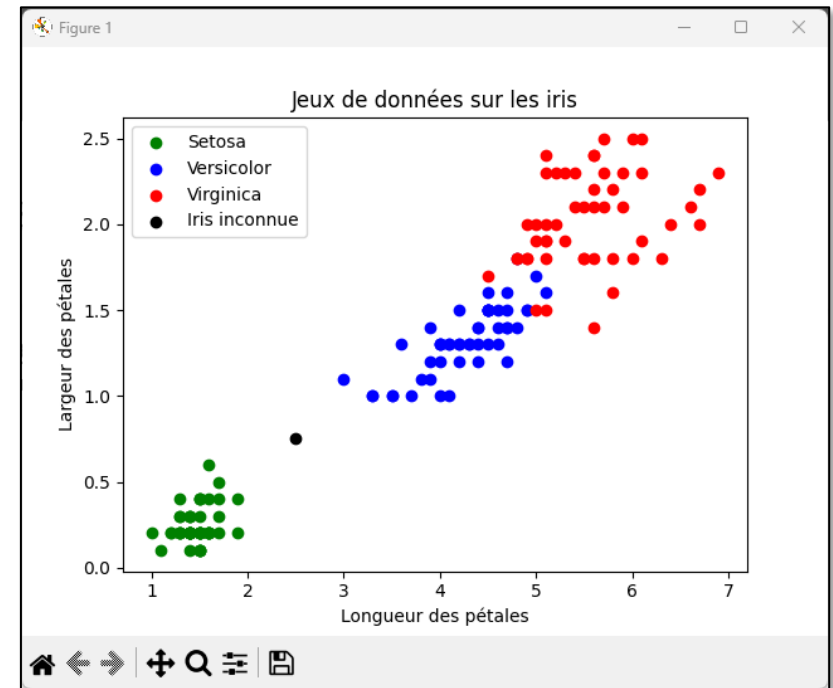
- Ajout de la nouvelle iris :

- Puis insérons la ligne permettant d'afficher cette nouvelle iris dans notre graphique :

```
28 # Affichage avec matplotlib :
29 # -----
30 plt.title("Jeux de données sur les iris")
31 plt.xlabel("Longueur des pétales")
32 plt.ylabel("Largeur des pétales")
33 plt.scatter(longueur_petale[espece == "setosa"], largeur_petale[espece == "setosa"], color = "green", label = "Setosa")
34 plt.scatter(longueur_petale[espece == "versicolor"], largeur_petale[espece == "versicolor"], color = "blue", label = "Versicolor")
35 plt.scatter(longueur_petale[espece == "virginica"], largeur_petale[espece == "virginica"], color = "red", label = "Virginica")
36 plt.scatter(iris_inconnue["longueur_pétale"], iris_inconnue["largeur_pétale"], color = "black", label = "Iris inconnue")
37 plt.legend()
38 plt.show()
39
```

Classification d'une iris inconnue

- Ajout de la nouvelle iris :
 - Exécutons le code pour constater la position de notre nouvelle iris.
 - L'iris est située au centre de la séparation des espèces *setosa* et *versicolor* !
 - Il nous est donc impossible à l'œil nu de déterminer son espèce.
 - La solution sera justement d'utiliser l'algorithme **K-NN** pour procéder au classement !



Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- Importons la classe **K-NN** de **Scikit-learn** à notre script Python :

```
1 # Import des bibliothèques :  
2 # -----  
3 import pandas as pd # On utilisera l'abréviation pd pour utiliser Pandas dans notre script  
4 import matplotlib.pyplot as plt # On utilisera l'abréviation plt pour utiliser matplotlib  
5 from sklearn.neighbors import KNeighborsClassifier as Knn # Import de la classe  
6
```

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- Il faut maintenant préparer les données d'entraînement (notre jeu de données) pour qu'il soit exploitable par **Scikit-learn**.
- Notre problème est que nous jusqu'à présent placé les données discriminatoires du jeu de test dans 2 listes/séries séparées : **longueur_petale** et **largeur_petale**.
- **Scikit-learn** attend à recevoir une liste de tuples, ou chaque tuple représente une iris du jeu de données :
 - ex: [(1.4, 0.2), (1.3, 0.2), (1.4, 0.2)] où (1.4, 0.2) représente une iris ayant une longueur de pétale de 1,4 cm et de 0,2 cm de largeur.
- Heureusement pour nous, il existe une fonction généralement appelée **zip()** qui est disponible dans beaucoup de langage de programmation, dont Python.

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- La fonction `zip()` prend en argument un ensemble de listes/tuples et renvoie une liste contenant des tuples avec les différentes valeurs données dans la suite de listes/tuples.
- Exemple illustrant le fonctionnement de la fonction `zip()` :
 - `list(zip((1, 2, 3), ("un", "deux", "trois"), ("one", "two", "three")))`
 - donnera la liste : `[(1, 'un', 'one'), (2, 'deux', 'two'), (3, 'trois', 'three')]`

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- Appliquons la fonction `zip()` pour créer une variable `zip_iris` qui contient le jeu de données pour **Scikit-learn** :

```
29 # Classification du nouvel individu :  
30 # -----  
31 zip_iris = list(zip(longueur_petale, largeur_petale)) # Mise en forme du jeux de données des longueurs et largeurs  
32
```

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- Nous allons maintenant instancier un modèle à partir de la classe **Knn**.
- L'instance attend à recevoir le nombre de voisins **k** sur lequel il devra appuyer les futures classifications.
- Rajouter le code suivant permettant la création de notre modèle avec **k = 3** :

```
33 k = 3 # Nombre de voisins validant la classification d'un nouveau individu
34 model = Knn(n_neighbors = k) # Instance d'un modèle
```

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- L'avant dernière étape est de procéder à l'entraînement de notre modèle avec le jeu de données.
- Pour ce faire, nous utilisons la méthode `fit()` à qui on fournira nos données (valeurs + étiquettes des espèces).
- Rajouter l'entraînement à votre code :

```
33 k = 3 # Nombre de voisins validant la classification d'un nouveau individu
34 model = Knn(n_neighbors = k) # Instance d'un modèle
35
36 model.fit(zip_iris, espece) # Entraînement du modèle
37
```

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- La dernière étape est de demander à notre modèle de classer notre nouvelle iris.
- Nous utilisons la méthode **predict()** qui prend en argument une liste de tuples où chaque tuple représente un individu à classer.
- La méthode **predict()** nous retournera une collection sur laquelle nous pourrions accéder aux différents individus classés par leur numéro d'indexation.
- Dans un premier temps nous afficherons le résultat de la classification directement dans la console.

```
36 model.fit(zip_iris, espece) # Entraînement du modèle
37
38 classification = model.predict([(iris_inconnue["longueur_pétale"], iris_inconnue["largeur_pétale"])]))
39 print(f"L'iris inconnue est classée (avec k={k}) comme : {classification[0]}") # Affichage du résultat dans la console
40
```

Classification d'une iris inconnue

- Utilisation de K-NN avec Scikit-learn :

- L'exécution du script doit maintenant afficher le résultat suivant dans la console :

```
>>> %Run '01-Classification iris.py'

   sepal_length  sepal_width  petal_length  petal_width  species
0             5.1           3.5           1.4           0.2    setosa
1             4.9           3.0           1.4           0.2    setosa
2             4.7           3.2           1.3           0.2    setosa
3             4.6           3.1           1.5           0.2    setosa
4             5.0           3.6           1.4           0.2    setosa
..           ...           ...           ...           ...     ...
145            6.7           3.0           5.2           2.3  virginica
146            6.3           2.5           5.0           1.9  virginica
147            6.5           3.0           5.2           2.0  virginica
148            6.2           3.4           5.4           2.3  virginica
149            5.9           3.0           5.1           1.8  virginica

[150 rows x 5 columns]
L'iris inconnue est classée (avec k=3) comme : setosa
```


Classification d'une iris inconnue

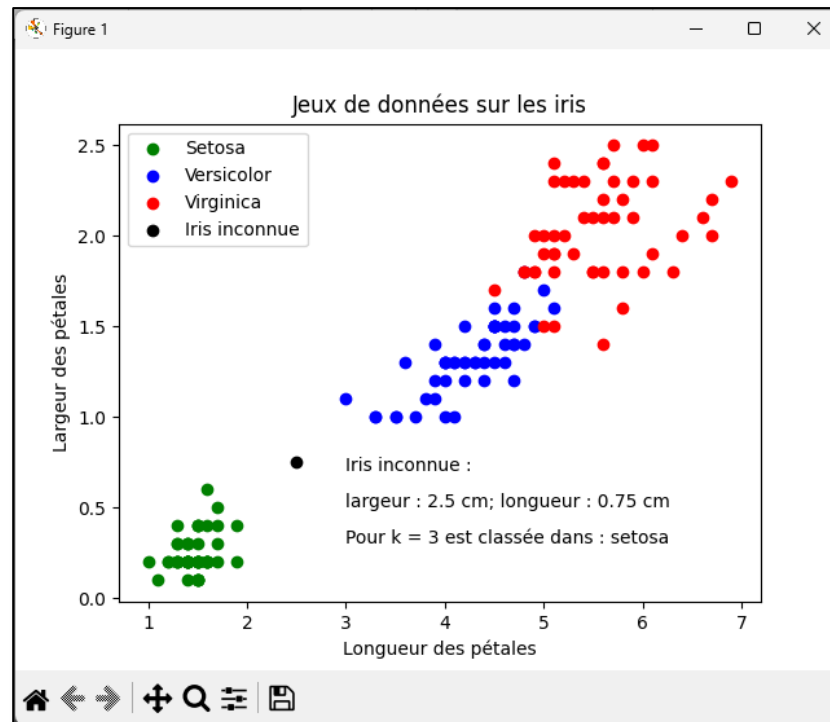
- Affichage du résultat directement sur le graphe :

- Nous allons également afficher le résultat de la classification directement dans le graphe.
- Pour ce faire rajouter les 3 lignes suivantes :

```
42 # Affichage avec matplotlib :
43 # -----
44 plt.title("Jeux de données sur les iris")
45 plt.xlabel("Longueur des pétales")
46 plt.ylabel("Largeur des pétales")
47 plt.scatter(longueur_petale[espece == "setosa"], largeur_petale[espece == "setosa"], color = "green", label = "Setosa")
48 plt.scatter(longueur_petale[espece == "versicolor"], largeur_petale[espece == "versicolor"], color = "blue", label = "Versicolor")
49 plt.scatter(longueur_petale[espece == "virginica"], largeur_petale[espece == "virginica"], color = "red", label = "Virginica")
50 plt.scatter(iris_inconnue["longueur_pétale"], iris_inconnue["largeur_pétale"], color = "black", label = "Iris inconnue")
51 plt.text(3, 0.7, "Iris inconnue :", fontsize = 10)
52 plt.text(3, 0.5, f"largeur : {iris_inconnue['longueur_pétale']} cm; longueur : {iris_inconnue['largeur_pétale']} cm", fontsize = 10)
53 plt.text(3, 0.3, f"Pour k = {k} est classée dans : {classification[0]}", fontsize = 10)
54 plt.legend()
55 plt.show()
```

Classification d'une iris inconnue

- Affichage du résultat directement sur le graphe :
 - Vous devriez avoir le résultat suivant :



Classification d'une iris inconnue

- Bilan :

- Vous avez avec 1^{er} exemple de classification :
 - Importé un jeu de données au format CSV avec **Pandas**
 - Représenté le jeu de données sous la forme d'un graphe en nuage de points avec **matplotlib**
 - Appliquer une classification avec l'algorithme **K-NN** au moyen de la bibliothèque **Scikit-learn**

Réinvestissement



Classification d'un personnage d'héroïc fantasy

Présentation

- **L'univers Héroïc Fantasy :**

- L'héroïc fantasy (ou « fantasy héroïque » en français) est un sous-genre de la fantasy qui plonge le lecteur dans des mondes imaginaires peuplés de héros, de magie, de créatures mythiques et de quêtes épiques. Inspiré par les mythes anciens, les légendes médiévales et les contes merveilleux, cet univers mêle souvent le merveilleux et l'aventure.

- **Des créatures mythiques :**

- L'héroïc fantasy foisonne d'êtres fantastiques : elfes immortels, nains forgerons, orcs sanguinaires, mais aussi licornes, griffons et bien d'autres. Ces peuples ajoutent une dimension mythologique à l'aventure.



Présentation

- La classification appliquée aux humains, nains et orcs :

- Nous vous proposons de prendre comme cas d'étude cet univers. L'idée est de déterminer la classification d'un personnage dont la race est inconnue et devra finalement être identifiée comme **humain**, **nain** ou **orc**.
- On utilisera 2 grandeurs de prédictions :
 - La taille
 - La masse



nain



humain



orc

Présentation

- La classification appliquée aux humains, nains et orcs :

- **Domaine de valeur masse et taille des 3 races :**

- **1. Nain :**

Petit mais robuste, le nain affiche une carrure trapue et puissante. Sa taille modeste (autour de **135 cm**) est compensée par une masse musculaire dense, et un large torse renforcé par une armure lourde. Sa barbe épaisse et soigneusement tressée, typique de son peuple, accentue son apparence déterminée et rustique.



- **2. Humain**

De stature moyenne (environ **175 cm**), l'humain présente une silhouette équilibrée et agile. Ses vêtements simples et sa musculature modérée en font un individu adaptable, taillé autant pour la survie que pour l'apprentissage. Son regard direct reflète une volonté propre à son espèce : la polyvalence.



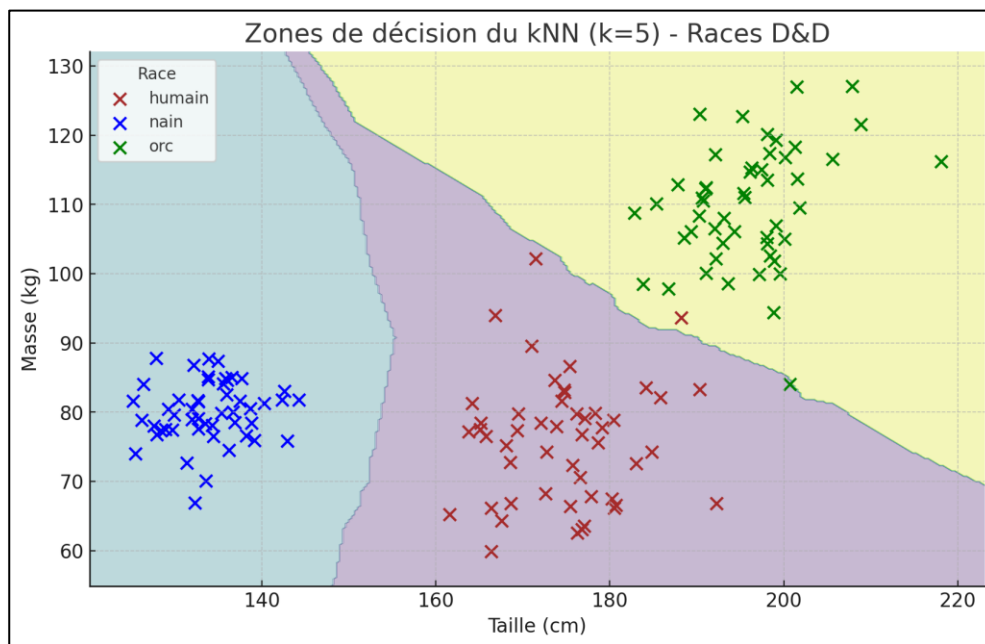
- **3. Orc :**

Imposant et massif, l'orc mesure facilement plus de **190 cm**. Son corps est taillé pour la force brute : épaules larges, bras puissants, mâchoire carrée. Sa peau légèrement verdâtre ou terreuse et son allure farouche ne laissent aucun doute sur ses origines belliqueuses.



Présentation

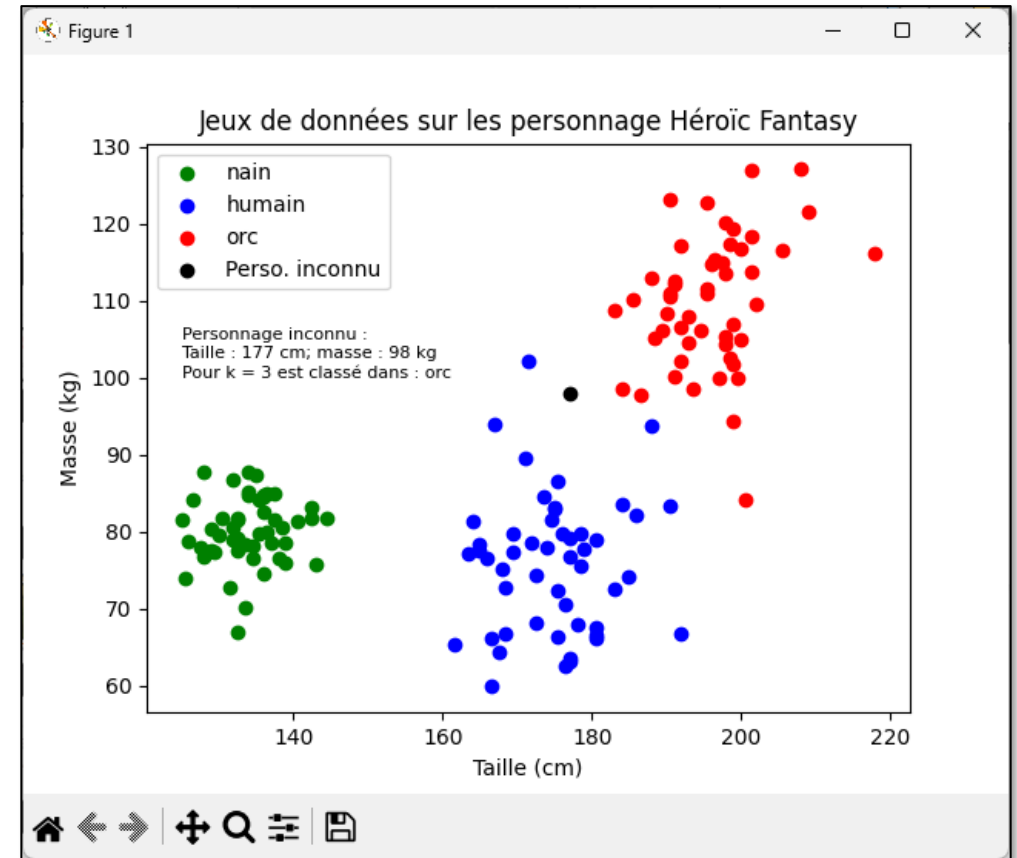
- La classification appliquée aux humains, nains et orcs :
 - Nous avons à notre disposition un jeu de données s'appuyant sur 3 populations de nains, humains et orcs :



Travail demandé

- Tâches à réaliser et objectifs :

- Copier le fichier **01-Classification iris.py** et renommer la copie en **02-Classification HeroicFantasy.py**
- Apporter l'ensemble des modifications nécessaires pour que la nouvelle version du fichier puisse :
 - Afficher le graphique en nuage de points du dataset
 - Déterminer un personnage inconnu d'une **taille de 177 cm** et d'une **masse de 98 kg**.



Travail demandé

- Ajout d'un prompt interactif :

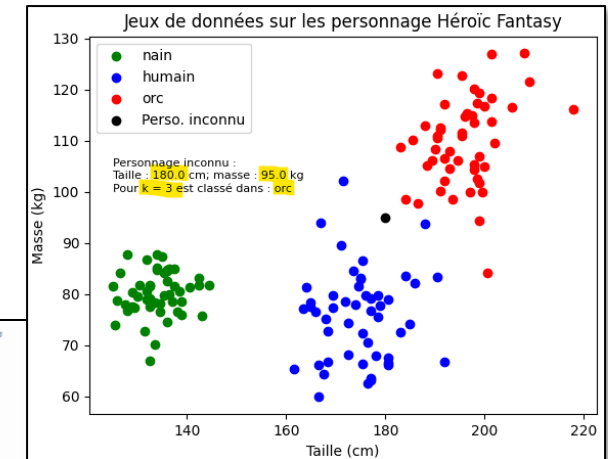
- Dans notre version actuelle de votre script, les données (taille et masse) sont inscrites en dur dans le code.
- Nous vous demandons d'apporter les modifications permettant la saisie via la console de la taille, de la masse et du nombre de voisins k pour procéder à la classification d'un personnage.
- Vous avez un exemple du résultat attendu ci-contre. ➡

Faire valider votre travail par le professeur

```
>>> %Run '02-Classification HeroicFantasy.py'

   taille  masse  race  race_encoded
0    137.5   81.6  nain             1
1    134.5   78.1  nain             1
2    138.0   76.6  nain             1
3    142.5   83.1  nain             1
4    134.0   85.2  nain             1
..      ...     ...     ...
145   193.0  104.5  orc              2
146   192.0  117.2  orc              2
147   191.0  112.5  orc              2
148   205.5  116.5  orc              2
149   197.5  115.0  orc              2

[150 rows x 4 columns]
Saisir la taille de votre personnage en cm : 180
Saisir la masse de votre personnage en kg : 95
Choisir le nombre de voisin k : 3
Le personnage inconnu est classé (avec k=3) comme : orc
```



Fin